Print copy of official Microsoft Learn documentation

Source: https://learn.microsoft.com/en-us/aspnet/core/tutorials/min-web-api?view=aspnetcore-10.0
Exported: 2026-10-14. Source metadata date: 2026-06-28T00:00:00Z.

Microsoft and documentation contributors. Article wording and code for the requested version are retained; web navigation is removed and print formatting is added. This locally generated PDF is not a Microsoft-published PDF book or a complete product manual. Source-page attribution and notices apply.

Documentation license and terms: Microsoft Learn terms; .NET documentation: CC BY 4.0 license. Check the source for specific attribution or exceptions.

Tutorial: Create a Minimal API with ASP.NET Core

By Wade Pickett and Tom Dykstra

Minimal APIs are designed to create HTTP APIs with minimal dependencies. They're ideal for microservices and apps that want to include only the minimum files, features, and dependencies in ASP.NET Core.

This tutorial teaches the basics of building a Minimal API with ASP.NET Core. Another approach to creating APIs in ASP.NET Core is to use controllers. For help with choosing between Minimal APIs and controller-based APIs, see APIs overview. For a tutorial on creating an API project based on controllers that contains more features, see Create a web API.

Overview

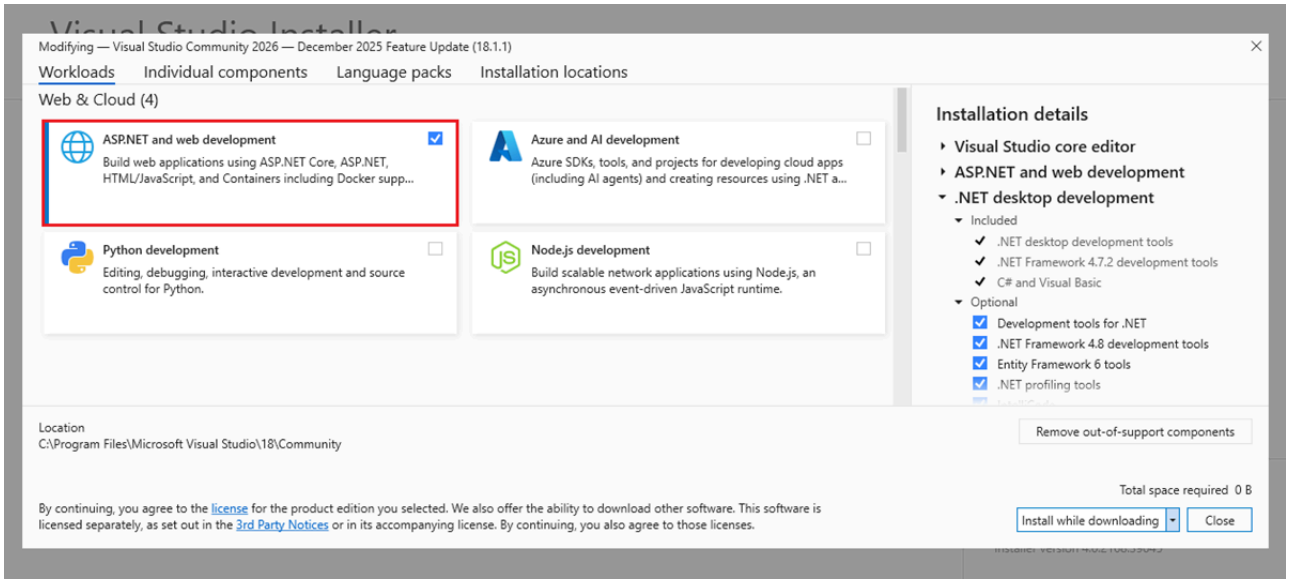
This tutorial creates the following API:

API	Description	Request body	Response body
`GET /todoitems`	Get all to-do items	None	Array of to-do items
`GET /todoitems/complete`	Get completed to-do items	None	Array of to-do items
`GET /todoitems/{id}`	Get an item by ID	None	To-do item
`POST /todoitems`	Add a new item	To-do item	To-do item
`PUT /todoitems/{id}`	Update an existing item	To-do item	None
`PATCH /todoitems/{id}`	Partially update an item	Partial to-do item	None
`DELETE /todoitems/{id}`	Delete an item	None	None

Prerequisites

- Visual Studio
- Visual Studio Code

- The latest version of Visual Studio with the **ASP.NET and web development** workload.



Create an API project

- Visual Studio
- Visual Studio Code

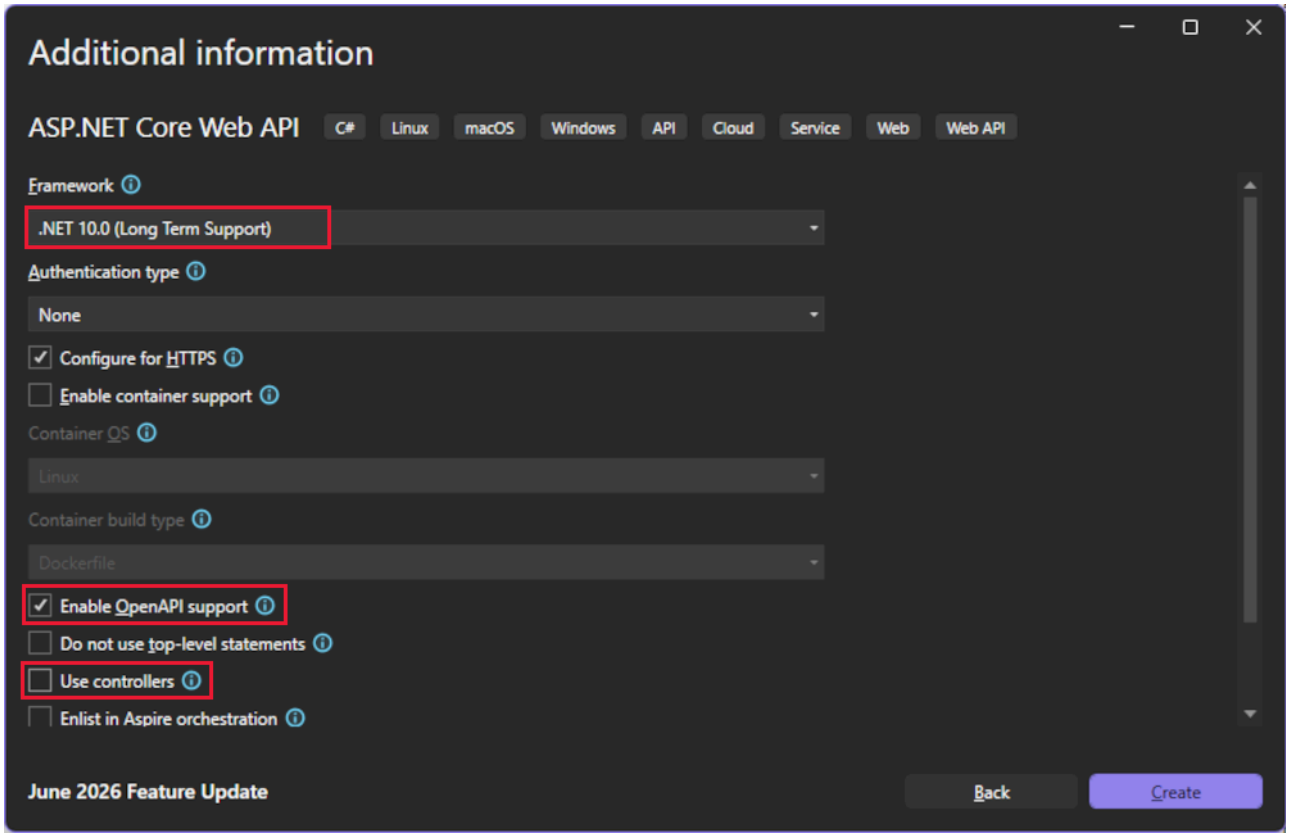
- Start Visual Studio 2026 and select **Create a new project**.

- In the **Create a new project** dialog:

  - Select the **ASP.NET Core Web API** project type, and select **Next**.
  - Name the project *TodoApi*, and select **Next**.

- In the **Additional information** dialog:

  - Confirm the **Framework** is **.NET 10.0 (Long Term Support)**.
  - Confirm the checkbox for **Enable OpenAPI support** is checked.
  - Confirm the checkbox for **Use controllers** is **not** checked. Uncheck this setting to create a Minimal API project as required for this tutorial rather than a controller-based one.
  - Select **Create**.



Examine the code

The `Program.cs` file generated by the template contains the following code:

- Visual Studio
- Visual Studio Code

```
var builder = WebApplication.CreateBuilder(args);

// Add services to the container.
// Learn more about configuring OpenAPI at https://aka.ms/aspnet/openapi
builder.Services.AddOpenApi();

var app = builder.Build();

// Configure the HTTP request pipeline.
if (app.Environment.IsDevelopment())
{
    app.MapOpenApi();
}

app.UseHttpsRedirection();

var summaries = new[]
{
    "Freezing", "Bracing", "Chilly", "Cool", "Mild", "Warm", "Balmy", "Hot", "Sweltering", "Scorching"
};

app.MapGet("/weatherforecast", () =>
{
    var forecast = Enumerable.Range(1, 5).Select(index =>
        new WeatherForecast
        (
```

```
            DateOnly.FromDateTime(DateTime.Now.AddDays(index)),
            Random.Shared.Next(-20, 55),
            summaries[Random.Shared.Next(summaries.Length)]
        ))
        .ToArray();
    return forecast;
})
.WithName("GetWeatherForecast");

app.Run();

internal record WeatherForecast(DateOnly Date, int TemperatureC, string? Summary)
{
    public int TemperatureF => 32 + (int)(TemperatureC / 0.5556);
}
```

The preceding code:

- Creates a WebApplicationBuilder and a WebApplication with preconfigured defaults.
- Registers the in-box OpenAPI document generator by using `builder.Services.AddOpenApi()`.
- Maps the generated OpenAPI document at `/openapi/v1.json` by using `app.MapOpenApi()`, in the Development environment only.
- Defines a sample `GET /weatherforecast` endpoint that returns five randomly generated `WeatherForecast` records.

In this tutorial, you replace the `WeatherForecast` with a new Todo sample, with endpoints to create, read, update, and delete items, backed by a model and a database.

Add NuGet packages

Add NuGet packages to support the database used in this tutorial.

- Visual Studio
- Visual Studio Code

- From the **Tools** menu, select **NuGet Package Manager > Manage NuGet Packages for Solution**.
- Select the **Browse** tab.
- Enter **Microsoft.EntityFrameworkCore.InMemory** in the search box, and then select `Microsoft.EntityFrameworkCore.InMemory`.
- Select the **Project** checkbox in the right pane and then select **Install**.

The model and database context classes

- In the project folder, create a file named `Todo.cs` with the following code:

```
public class Todo
{
    public int Id { get; set; }
    public string? Name { get; set; }
    public bool IsComplete { get; set; }
    public string? Secret { get; set; }
}
```

The preceding code creates the model for this app. A *model* is a class that represents data that the app manages.

The `Secret` property is included to demonstrate a common real-world need: data the app stores and uses internally, such as the ID of the user who owns the item, that you don't want clients to see or set. In the Prevent over-posting step later in this tutorial, you use a Data Transfer Object (DTO) to keep fields like `Secret` out of the API's input and responses.

- Create a file named `TodoDb.cs` with the following code:

```
using Microsoft.EntityFrameworkCore;

class TodoDb : DbContext
{
    public TodoDb(DbContextOptions<TodoDb> options)
        : base(options) { }

    public DbSet<Todo> Todos => Set<Todo>();
}
```

The preceding code defines the *database context*, which is the main class that coordinates Entity Framework functionality for a data model. This class derives from the Microsoft.EntityFrameworkCore.DbContext class.

The app uses an in-memory database named `TodoList`. The database is registered later in `Program.cs` with `builder.Services.AddDbContext<TodoDb>(opt => opt.UseInMemoryDatabase("TodoList"));`, where the string `"TodoList"` is the database name. Because the data is stored in memory, it's reset every time the app restarts.

Replace the `WeatherForecast` sample with the Todo API

The `webapi` template adds a sample `GET /weatherforecast` endpoint to `Program.cs` and a `WeatherForecast` record at the bottom of the file. The sample is just a placeholder to show the template works - replace both with the Todo endpoints described in the following section.

- Replace all of the code in `Program.cs` with the following. The `WeatherForecast` sample endpoint and record are removed, and the Todo endpoints take their place:

- Visual Studio
- Visual Studio Code

```
using Microsoft.EntityFrameworkCore;

var builder = WebApplication.CreateBuilder(args);
builder.Services.AddDbContext<TodoDb>(opt => opt.UseInMemoryDatabase("TodoList"));
builder.Services.AddOpenApi();
var app = builder.Build();

if (app.Environment.IsDevelopment())
{
    app.MapOpenApi();
}

app.MapGet("/todoitems", async (TodoDb db) =>
    await db.Todos.ToListAsync());
```

```
app.MapGet("/todoitems/complete", async (TodoDb db) =>
    await db.Todos.Where(t => t.IsComplete).ToListAsync());

app.MapGet("/todoitems/{id}", async (int id, TodoDb db) =>
    await db.Todos.FindAsync(id)
        is Todo todo
            ? Results.Ok(todo)
            : Results.NotFound());

app.MapPost("/todoitems", async (Todo todo, TodoDb db) =>
{
    db.Todos.Add(todo);
    await db.SaveChangesAsync();

    return Results.Created($"/todoitems/{todo.Id}", todo);
});

app.MapPut("/todoitems/{id}", async (int id, Todo inputTodo, TodoDb db) =>
{
    var todo = await db.Todos.FindAsync(id);

    if (todo is null) return Results.NotFound();

    todo.Name = inputTodo.Name;
    todo.IsComplete = inputTodo.IsComplete;

    await db.SaveChangesAsync();

    return Results.NoContent();
});

app.MapDelete("/todoitems/{id}", async (int id, TodoDb db) =>
{
    if (await db.Todos.FindAsync(id) is Todo todo)
    {
        db.Todos.Remove(todo);
        await db.SaveChangesAsync();
        return Results.NoContent();
    }

    return Results.NotFound();
});

app.Run();
```

The OpenAPI lines that the template adds (`builder.Services.AddOpenApi()` and `app.MapOpenApi()`) stay in place. They now describe the Todo endpoints instead of the sample weather endpoint.

The following highlighted code registers the app's services in the dependency injection (DI) container: the database context (`AddDbContext`) and the OpenAPI document generator (`AddOpenApi`):

- Visual Studio
- Visual Studio Code

```
var builder = WebApplication.CreateBuilder(args);
builder.Services.AddDbContext<TodoDb>(opt => opt.UseInMemoryDatabase("TodoList"));
builder.Services.AddOpenApi();
var app = builder.Build();
```

The DI container provides access to the database context and other services.

- Visual Studio
- Visual Studio Code

This tutorial uses Endpoints Explorer and .http files to test the API.

Test posting data

The following code in `Program.cs` creates an HTTP POST endpoint `/todoitems` that adds data to the in-memory database:

- Visual Studio
- Visual Studio Code

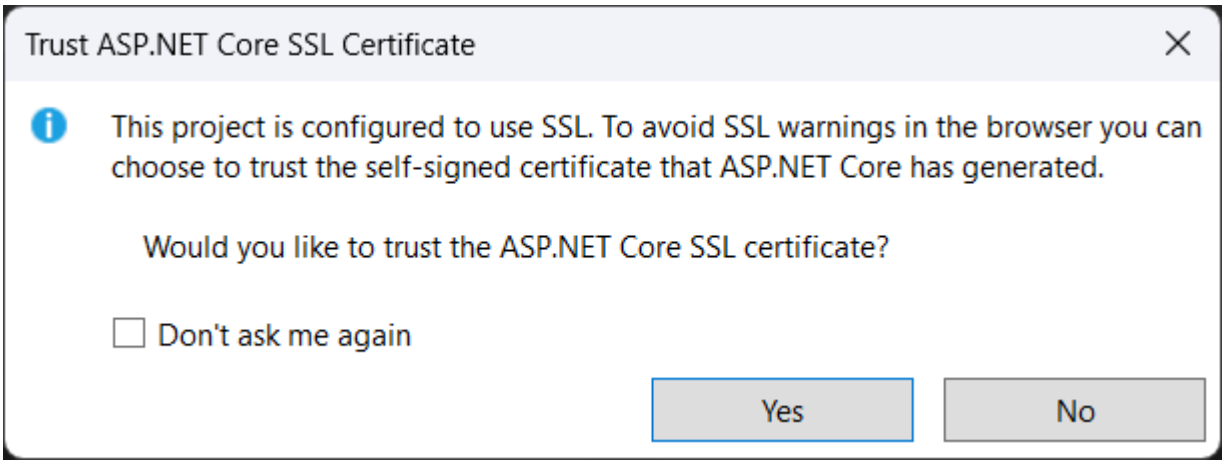
```
app.MapPost("/todoitems", async (Todo todo, TodoDb db) =>
{
    db.Todos.Add(todo);
    await db.SaveChangesAsync();

    return Results.Created($"/todoitems/{todo.Id}", todo);
});
```

- Visual Studio
- Visual Studio Code

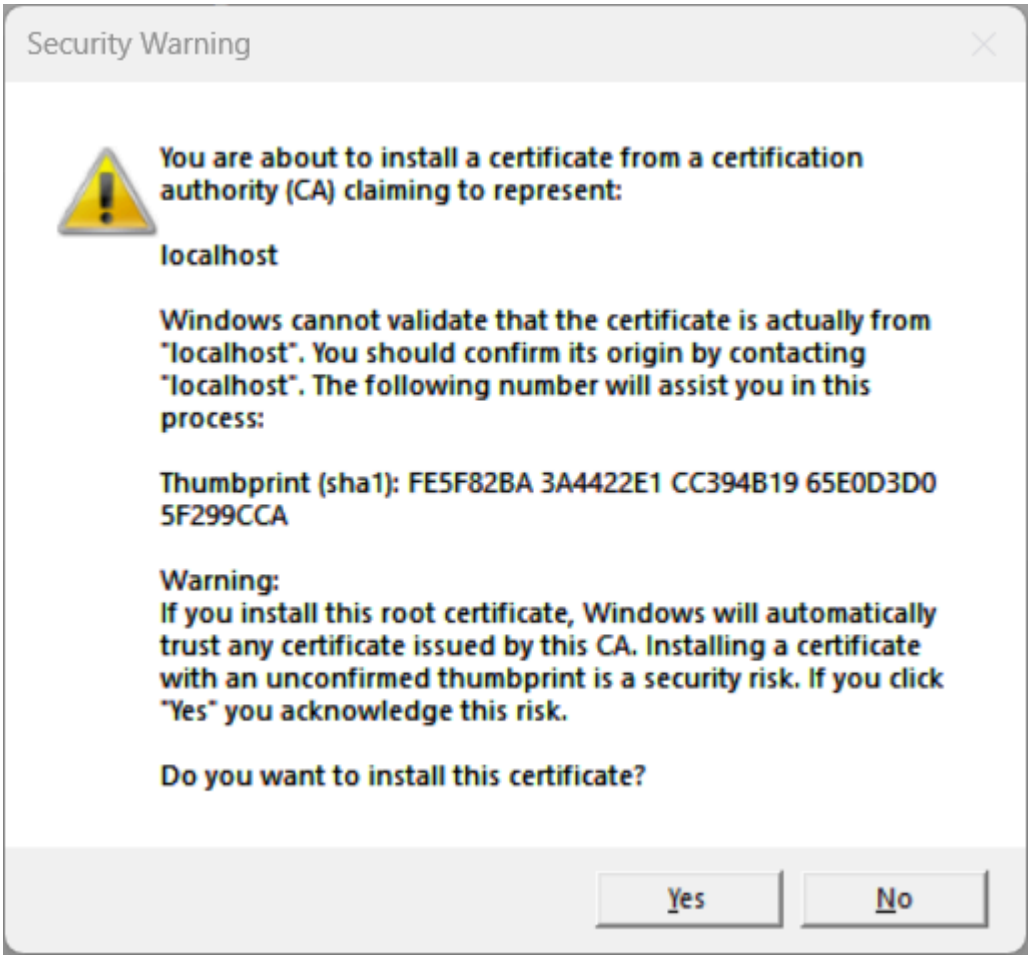
- Press `Ctrl+F5` to run the app without debugging. Visual Studio launches the Kestrel web server and trusts the development certificate if needed.

Visual Studio displays the following dialog:



Select **Yes** if you trust the IIS Express SSL certificate.

The following dialog is displayed:

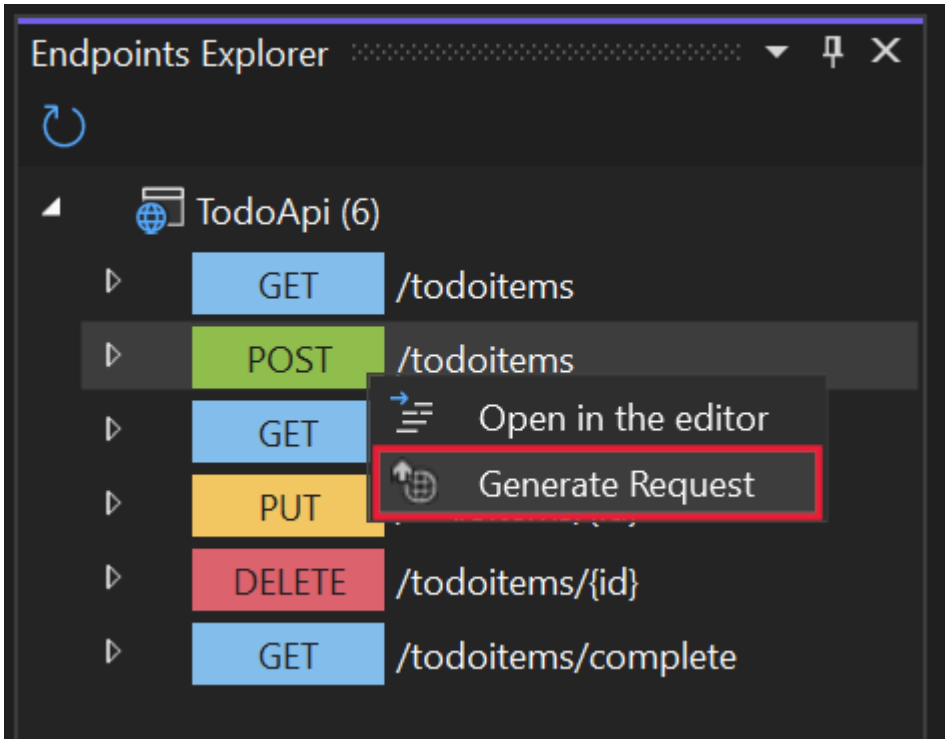


Select **Yes** if you agree to trust the development certificate.

For information on trusting the Firefox browser, see Firefox SEC_ERROR_INADEQUATE_KEY_USAGE certificate error.

Use the POST endpoint to add data to the app.

- Select **View** > **Other Windows** > **Endpoints Explorer**.
- Right-click the **POST** endpoint and select **Generate request**.



A new file is created in the project folder named `TodoApi.http`, with contents similar to the following example:

```
@TodoApi_HostAddress = https://localhost:7031

POST {{TodoApi_HostAddress}}/todoitems

###
```

- The first line creates a variable that is used for all of the endpoints.
- The next line defines a POST request.
- The triple hashtag (###) line is a request delimiter: what comes after it is for a different request.
- The POST request needs headers and a body. To define those parts of the request, add the following lines immediately after the POST request line:

```
Content-Type: application/json

{
  "name":"walk dog",
  "isComplete":true
}
```

The preceding code adds a Content-Type header and a JSON request body. The TodoApi.http file should now look like the following example, but with your port number:

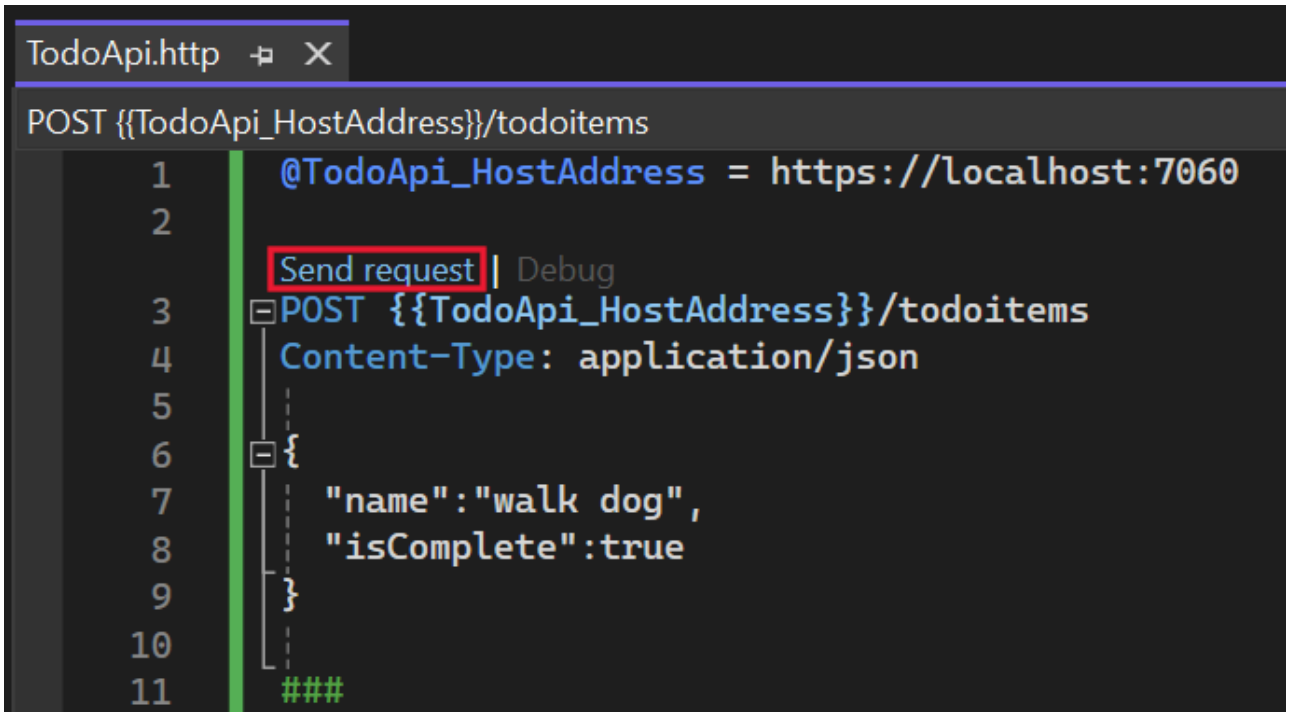
```
@TodoApi_HostAddress = https://localhost:7057

POST {{TodoApi_HostAddress}}/todoitems
Content-Type: application/json
```

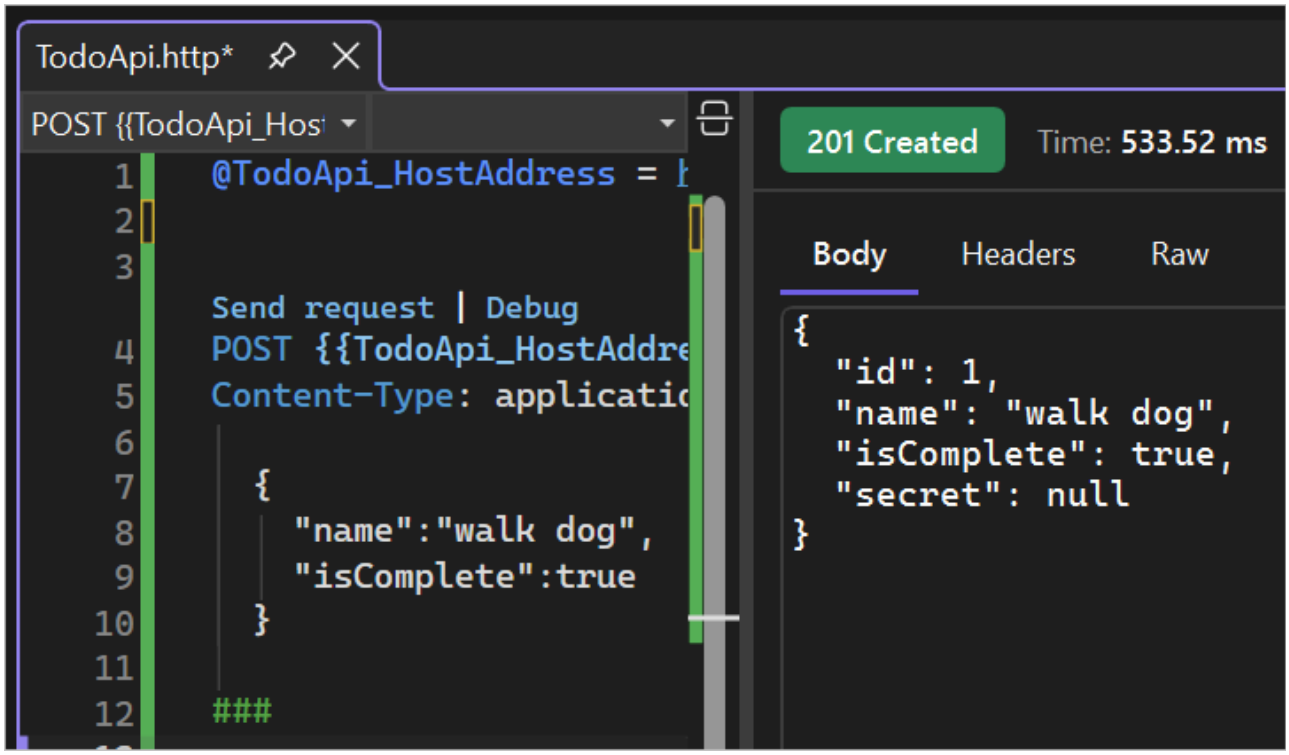
```
{
  "name":"walk dog",
  "isComplete":true
}

###
```

- Select the **Send request** link that is above the `POST` request line.



The POST request is sent to the app and the response is displayed in the **Response** pane.



Examine the GET endpoints

Your `Program.cs` includes several GET endpoints, defined with `MapGet`:

API	Description	Request body	Response body
`GET /todoitems`	Get all to-do items	None	Array of to-do items
`GET /todoitems/complete`	Get all completed to-do items	None	Array of to-do items
`GET /todoitems/{id}`	Get an item by ID	None	To-do item

```
app.MapGet("/todoitems", async (TodoDb db) =>
    await db.Todos.ToListAsync());

app.MapGet("/todoitems/complete", async (TodoDb db) =>
    await db.Todos.Where(t => t.IsComplete).ToListAsync());

app.MapGet("/todoitems/{id}", async (int id, TodoDb db) =>
    await db.Todos.FindAsync(id)
        is Todo todo
            ? Results.Ok(todo)
            : Results.NotFound());
```

Test the GET endpoints

- Visual Studio
- Visual Studio Code

Test the app by calling the `GET` endpoints from a browser or by using **Endpoints Explorer**. The following steps are for **Endpoints Explorer**.

- In **Endpoints Explorer**, right-click the first **GET** endpoint, and select **Generate request**.

  The following content is added to the `TodoApi.http` file:

```
GET {{TodoApi_HostAddress}}/todoitems

###
```

- Select the **Send request** link that is above the new `GET` request line.

  The GET request is sent to the app and the response is displayed in the **Response** pane.

- The response body is similar to the following JSON:

```
[
  {
    "id": 1,
    "name": "walk dog",
    "isComplete": true,
    "secret": null
  }
]
```

- In **Endpoints Explorer**, right-click the `/todoitems/{id}` **GET** endpoint and select **Generate request**. The following content is added to the `TodoApi.http` file:

```
GET {{TodoApi_HostAddress}}/todoitems/{id}

###
```

- Replace `{id}` with `1`.
- Select the **Send request** link that is above the new GET request line.

  The GET request is sent to the app and the response is displayed in the **Response** pane.
- The response body is similar to the following JSON:

```
{
  "id": 1,
  "name": "walk dog",
  "isComplete": true,
  "secret": null
}
```

This app uses an in-memory database. If you restart the app, the data is lost: `GET /todoitems` returns an empty array (`[]`), and `GET /todoitems/{id}` returns a `404 Not Found`. To repopulate the app, POST data to the app and try the GET request again.

Return values

ASP.NET Core automatically serializes the object to JSON and writes the JSON into the body of the response message. The response code for this return type is 200 OK, assuming there are no unhandled exceptions. Unhandled exceptions are translated into 5xx errors.

The return types can represent a wide range of HTTP status codes. For example, `GET /todoitems/{id}` can return two different status values:

- If no item matches the requested ID, the method returns a 404 status NotFound error code.
- Otherwise, the method returns 200 with a JSON response body. Returning `item` results in an HTTP 200 response.

Examine the PUT endpoint

The sample app implements a single PUT endpoint using `MapPut`:

- Visual Studio
- Visual Studio Code

```
app.MapPut("/todoitems/{id}", async (int id, Todo inputTodo, TodoDb db) =>
{
    var todo = await db.Todos.FindAsync(id);

    if (todo is null) return Results.NotFound();
```

```
    todo.Name = inputTodo.Name;
    todo.IsComplete = inputTodo.IsComplete;

    await db.SaveChangesAsync();

    return Results.NoContent();
});
```

This method is similar to the `MapPost` method, except it uses HTTP PUT. A successful response returns 204 (No Content). According to the HTTP specification, a PUT request requires the client to send the entire updated entity, not just the changes. To support partial updates, use HTTP PATCH.

Test the PUT endpoint

This sample uses an in-memory database that you must initialize each time you start the app. You need an item in the database before you make a PUT call. Call POST to ensure there's an item in the database before making a PUT call.

Update the Todo item that has `Id = 1` and set its name to `"feed fish"`.

- Visual Studio
- Visual Studio Code

- In **Endpoints Explorer**, right-click the **PUT** endpoint, and select **Generate request**.

  The following content is added to the `TodoApi.http` file:

```
PUT {{TodoApi_HostAddress}}/todoitems/{id}

###
```

- In the PUT request line, replace `{id}` with `1`.
- Add the following lines immediately after the PUT request line:

```
Content-Type: application/json

{
  "id": 1,
  "name": "feed fish",
  "isComplete": false
}
```

The preceding code adds a Content-Type header and a JSON request body.

- Select the **Send request** link that is above the new PUT request line.

The PUT request is sent to the app and the response is displayed in the **Response** pane. The response body is empty, and the status code is 204.

Create and examine the PATCH endpoint

A PATCH endpoint lets clients send only the fields they want to update, such as renaming a Todo item without resending its completion status. This approach differs from a PUT request, which replaces the entire item, so the client must send every field even if only one is changed.

The next steps add a new file and modify the `Program.cs` file. Stop the `TodoApi` app before making these changes. Leave the Scalar page in the browser running.

- Visual Studio
- Visual Studio Code

- To stop the app, select the **Stop** button (the red square) in the Visual Studio toolbar.

This sample uses an in-memory database that you must initialize each time the app starts. The database must contain an item before you make a PATCH call. Call POST to ensure the database has an item before making a PATCH call.

The PATCH endpoint uses a `TodoPatchDto` class with nullable properties to properly handle partial updates. By using nullable properties, the endpoint can distinguish between a field that wasn't provided (null) and a field explicitly set to a value (including false for boolean fields). Without nullable properties, a non-nullable bool defaults to false, which could potentially overwrite an existing true value when that field isn't included in the request.

- Create a file named `TodoPatchDto.cs` with the following code:

```
public class TodoPatchDto
{
    public string? Name { get; set; }
    public bool? IsComplete { get; set; }
}
```

The `TodoPatchDto` class uses nullable properties (`string?` and `bool?`) to distinguish between a field that wasn't provided in the request versus a field explicitly set to a value.

- In `Program.cs`, add the following PATCH endpoint immediately after the `MapPut` endpoint:

- Visual Studio
- Visual Studio Code

```
app.MapPatch("/todoitems/{id}", async (int id, TodoPatchDto inputTodo, TodoDb db) =>
{
    var todo = await db.Todos.FindAsync(id);

    if (todo is null) return Results.NotFound();

    if (inputTodo.Name is not null) todo.Name = inputTodo.Name;
    if (inputTodo.IsComplete is not null) todo.IsComplete = inputTodo.IsComplete.Value;

    await db.SaveChangesAsync();

    return Results.NoContent();
});
```

This method is similar to the `MapPut` method, but it uses HTTP PATCH and only updates the fields provided in the request. A successful response returns 204 (No Content).

> Note
>
> PATCH operations allow partial updates to resources. For more advanced partial updates using JSON Patch documents, see JsonPatch in ASP.NET Core web API.

Test the PATCH endpoint

- Visual Studio
- Visual Studio Code

- Press `Ctrl+F5` to rebuild and run the app with the new PATCH endpoint.

This sample uses an in-memory database that you must initialize each time the app starts. The database must contain an item before you make a PATCH call. Call POST to ensure the database has an item before making a PATCH call.

Update only the `name` property of the Todo item that has `Id = 1` and set its name to `"run errands"`.

- Visual Studio
- Visual Studio Code

- In **Endpoints Explorer**, select the refresh button. Then, right-click the **PATCH** endpoint, and select **Generate request**.

  The following content is added to the `TodoApi.http` file:

```
PATCH {{TodoApi_HostAddress}}/todoitems/{id}

###
```

- In the PATCH request line, replace `{id}` with `1`.
- Add the following lines immediately after the PATCH request line:

```
Content-Type: application/json

{
  "name": "run errands"
}
```

The preceding code adds a Content-Type header and a JSON request body with only the field to update.

- Select the **Send request** link that is above the new PATCH request line.

The PATCH request is sent to the app and the response is displayed in the **Response** pane. The response body is empty, and the status code is 204.

Examine the DELETE endpoint

Your `Program.cs` file includes a single DELETE endpoint, defined with `MapDelete`:

- Visual Studio
- Visual Studio Code

```
app.MapDelete("/todoitems/{id}", async (int id, TodoDb db) =>
{
    if (await db.Todos.FindAsync(id) is Todo todo)
    {
        db.Todos.Remove(todo);
        await db.SaveChangesAsync();
        return Results.NoContent();
    }

    return Results.NotFound();
});
```

Test the DELETE endpoint

- Visual Studio
- Visual Studio Code

- In **Endpoints Explorer**, right-click the **DELETE** endpoint and select **Generate request**.

  A DELETE request is added to `TodoApi.http`.

- Replace `{id}` in the DELETE request line with `1`. The DELETE request should look like the following example:

```
DELETE {{TodoApi_HostAddress}}/todoitems/1

###
```

- Select the **Send request** link for the DELETE request.

The DELETE request is sent to the app and the response is displayed in the **Response** pane. The response body is empty, and the status code is 204.

Use the MapGroup API

- The next steps modify the `Program.cs` file, so stop the app before making these changes. Leave the scalar page in the browser running.

- Visual Studio
- Visual Studio Code

- To stop the app, select the **Stop** button (the red square) in the Visual Studio toolbar.

The `Program.cs` file you wrote repeats the `todoitems` URL prefix each time it sets up an endpoint. APIs often have groups of endpoints with a common URL prefix, and the MapGroup method helps organize such groups. It reduces repetitive code and allows you to customize entire groups of endpoints with a single call to methods like RequireAuthorization and WithMetadata.

- Replace the contents of `Program.cs` with the following code:

- Visual Studio
- Visual Studio Code

```
using Microsoft.EntityFrameworkCore;

var builder = WebApplication.CreateBuilder(args);
builder.Services.AddDbContext<TodoDb>(opt => opt.UseInMemoryDatabase("TodoList"));
builder.Services.AddOpenApi();
var app = builder.Build();

if (app.Environment.IsDevelopment())
{
    app.MapOpenApi();
}

var todoItems = app.MapGroup("/todoitems");

todoItems.MapGet("/", async (TodoDb db) =>
    await db.Todos.ToListAsync());

todoItems.MapGet("/complete", async (TodoDb db) =>
    await db.Todos.Where(t => t.IsComplete).ToListAsync());

todoItems.MapGet("/{id}", async (int id, TodoDb db) =>
    await db.Todos.FindAsync(id)
        is Todo todo
            ? Results.Ok(todo)
            : Results.NotFound());

todoItems.MapPost("/", async (Todo todo, TodoDb db) =>
{
    db.Todos.Add(todo);
    await db.SaveChangesAsync();

    return Results.Created($"/todoitems/{todo.Id}", todo);
});

todoItems.MapPut("/{id}", async (int id, Todo inputTodo, TodoDb db) =>
{
    var todo = await db.Todos.FindAsync(id);

    if (todo is null) return Results.NotFound();

    todo.Name = inputTodo.Name;
    todo.IsComplete = inputTodo.IsComplete;

    await db.SaveChangesAsync();

    return Results.NoContent();
});

todoItems.MapPatch("/{id}", async (int id, TodoPatchDto inputTodo, TodoDb db) =>
{
    var todo = await db.Todos.FindAsync(id);
```

```
    if (todo is null) return Results.NotFound();

    if (inputTodo.Name is not null) todo.Name = inputTodo.Name;
    if (inputTodo.IsComplete is not null) todo.IsComplete = inputTodo.IsComplete.Value;

    await db.SaveChangesAsync();

    return Results.NoContent();
});

todoItems.MapDelete("/{id}", async (int id, TodoDb db) =>
{
    if (await db.Todos.FindAsync(id) is Todo todo)
    {
        db.Todos.Remove(todo);
        await db.SaveChangesAsync();
        return Results.NoContent();
    }

    return Results.NotFound();
});

app.Run();
```

The preceding code has the following changes:

- Adds `var todoItems = app.MapGroup("/todoitems");` to set up the group using the URL prefix `/todoitems`.
- Changes all the `app.Map<HttpVerb>` methods to `todoItems.Map<HttpVerb>`.
- Removes the URL prefix `/todoitems` from the `Map<HttpVerb>` method calls.
- Runs the app and tests the endpoints to verify that they work the same.

Use the TypedResults API

The next steps modify the `Program.cs` file, so stop the app before making these changes. Leave the scalar page in the browser running.

Returning TypedResults rather than Results has several advantages, including testability and automatically returning the response type metadata for OpenAPI to describe the endpoint. For more information, see `TypedResults` versus `Results`.

- The `Map<HttpVerb>` methods can call route handler methods instead of using lambdas. To see an example, update *Program.cs* with the following code:

- Visual Studio
- Visual Studio Code

```
using Microsoft.EntityFrameworkCore;

var builder = WebApplication.CreateBuilder(args);
builder.Services.AddDbContext<TodoDb>(opt => opt.UseInMemoryDatabase("TodoList"));
builder.Services.AddOpenApi();
var app = builder.Build();
```

```
if (app.Environment.IsDevelopment())
{
    app.MapOpenApi();
}

var todoItems = app.MapGroup("/todoitems");

todoItems.MapGet("/", GetAllTodos);
todoItems.MapGet("/complete", GetCompleteTodos);
todoItems.MapGet("/{id}", GetTodo);
todoItems.MapPost("/", CreateTodo);
todoItems.MapPut("/{id}", UpdateTodo);
todoItems.MapPatch("/{id}", PatchTodo);
todoItems.MapDelete("/{id}", DeleteTodo);

app.Run();

static async Task<IResult> GetAllTodos(TodoDb db)
{
    return TypedResults.Ok(await db.Todos.ToArrayAsync());
}

static async Task<IResult> GetCompleteTodos(TodoDb db)
{
    return TypedResults.Ok(await db.Todos.Where(t => t.IsComplete).ToListAsync());
}

static async Task<IResult> GetTodo(int id, TodoDb db)
{
    return await db.Todos.FindAsync(id)
        is Todo todo
            ? TypedResults.Ok(todo)
            : TypedResults.NotFound();
}

static async Task<IResult> CreateTodo(Todo todo, TodoDb db)
{
    db.Todos.Add(todo);
    await db.SaveChangesAsync();

    return TypedResults.Created($"/todoitems/{todo.Id}", todo);
}

static async Task<IResult> UpdateTodo(int id, Todo inputTodo, TodoDb db)
{
    var todo = await db.Todos.FindAsync(id);

    if (todo is null) return TypedResults.NotFound();

    todo.Name = inputTodo.Name;
    todo.IsComplete = inputTodo.IsComplete;

    await db.SaveChangesAsync();

    return TypedResults.NoContent();
}

static async Task<IResult> PatchTodo(int id, TodoPatchDto inputTodo, TodoDb db)
{
    var todo = await db.Todos.FindAsync(id);

    if (todo is null) return TypedResults.NotFound();
```

```
    if (inputTodo.Name is not null) todo.Name = inputTodo.Name;
    if (inputTodo.IsComplete is not null) todo.IsComplete = inputTodo.IsComplete.Value;

    await db.SaveChangesAsync();

    return TypedResults.NoContent();
}

static async Task<IResult> DeleteTodo(int id, TodoDb db)
{
    if (await db.Todos.FindAsync(id) is Todo todo)
    {
        db.Todos.Remove(todo);
        await db.SaveChangesAsync();
        return TypedResults.NoContent();
    }

    return TypedResults.NotFound();
}
```

The `Map<HttpVerb>` code now calls methods instead of lambdas:

```
var todoItems = app.MapGroup("/todoitems");

todoItems.MapGet("/", GetAllTodos);
todoItems.MapGet("/complete", GetCompleteTodos);
todoItems.MapGet("/{id}", GetTodo);
todoItems.MapPost("/", CreateTodo);
todoItems.MapPut("/{id}", UpdateTodo);
todoItems.MapPatch("/{id}", PatchTodo);
todoItems.MapDelete("/{id}", DeleteTodo);
```

The following methods return objects that implement IResult and are defined by TypedResults:

```
static async Task<IResult> GetAllTodos(TodoDb db)
{
    return TypedResults.Ok(await db.Todos.ToArrayAsync());
}

static async Task<IResult> GetCompleteTodos(TodoDb db)
{
    return TypedResults.Ok(await db.Todos.Where(t => t.IsComplete).ToListAsync());
}

static async Task<IResult> GetTodo(int id, TodoDb db)
{
    return await db.Todos.FindAsync(id)
        is Todo todo
            ? TypedResults.Ok(todo)
            : TypedResults.NotFound();
}

static async Task<IResult> CreateTodo(Todo todo, TodoDb db)
{
    db.Todos.Add(todo);
    await db.SaveChangesAsync();

    return TypedResults.Created($"/todoitems/{todo.Id}", todo);
}

static async Task<IResult> UpdateTodo(int id, Todo inputTodo, TodoDb db)
```

```
{
    var todo = await db.Todos.FindAsync(id);

    if (todo is null) return TypedResults.NotFound();

    todo.Name = inputTodo.Name;
    todo.IsComplete = inputTodo.IsComplete;

    await db.SaveChangesAsync();

    return TypedResults.NoContent();
}

static async Task<IResult> PatchTodo(int id, TodoPatchDto inputTodo, TodoDb db)
{
    var todo = await db.Todos.FindAsync(id);

    if (todo is null) return TypedResults.NotFound();

    if (inputTodo.Name is not null) todo.Name = inputTodo.Name;
    if (inputTodo.IsComplete is not null) todo.IsComplete = inputTodo.IsComplete.Value;

    await db.SaveChangesAsync();

    return TypedResults.NoContent();
}

static async Task<IResult> DeleteTodo(int id, TodoDb db)
{
    if (await db.Todos.FindAsync(id) is Todo todo)
    {
        db.Todos.Remove(todo);
        await db.SaveChangesAsync();
        return TypedResults.NoContent();
    }

    return TypedResults.NotFound();
}
```

Unit tests can call these methods and test that they return the correct type. For example, if the method is `GetAllTodos`:

```
static async Task<IResult> GetAllTodos(TodoDb db)
{
    return TypedResults.Ok(await db.Todos.ToArrayAsync());
}
```

Unit test code can verify that an object of type Ok<Todo[]> is returned from the handler method. For example:

```
public async Task GetAllTodos_ReturnsOkOfTodosResult()
{
    // Arrange
    var db = CreateDbContext();

    // Act
    var result = await TodosApi.GetAllTodos(db);

    // Assert: Check for the correct returned type
    Assert.IsType<Ok<Todo[]>>(result);
}
```

Prevent over-posting

Currently, your API exposes the entire `Todo` object, including the `Secret` property you added in The model and database context classes. In production applications, use a subset of the model to restrict the data that clients can input and receive. Security is a major reason for this restriction. This subset of a model is usually referred to as a Data Transfer Object (DTO), input model, or view model. This article uses **DTO**.

Use a DTO to:

- Prevent over-posting.
- Hide properties that clients aren't supposed to view.
- Omit some properties to reduce payload size.
- Flatten object graphs that contain nested objects. Flattened object graphs can be more convenient for clients.

This app needs to hide the `Secret` field, but an administrative app could choose to expose it.

- The next steps add a file, so stop the app before making these changes. Leave the scalar page in the browser running.

- Create a file named `TodoItemDTO.cs` with the following code:

```
public class TodoItemDTO
{
    public int Id { get; set; }
    public string? Name { get; set; }
    public bool IsComplete { get; set; }

    public TodoItemDTO() { }
    public TodoItemDTO(Todo todoItem) =>
    (Id, Name, IsComplete) = (todoItem.Id, todoItem.Name, todoItem.IsComplete);
}
```

- Replace the contents of the `Program.cs` file with the following code to use this DTO model:

- Visual Studio
- Visual Studio Code

```
using Microsoft.EntityFrameworkCore;

var builder = WebApplication.CreateBuilder(args);
builder.Services.AddDbContext<TodoDb>(opt => opt.UseInMemoryDatabase("TodoList"));
builder.Services.AddOpenApi();
var app = builder.Build();

if (app.Environment.IsDevelopment())
{
    app.MapOpenApi();
}

var todoItems = app.MapGroup("/todoitems");

todoItems.MapGet("/", GetAllTodos);
todoItems.MapGet("/complete", GetCompleteTodos);
todoItems.MapGet("/{id}", GetTodo);
```

```
todoItems.MapPost("/", CreateTodo);
todoItems.MapPut("/{id}", UpdateTodo);
todoItems.MapPatch("/{id}", PatchTodo);
todoItems.MapDelete("/{id}", DeleteTodo);

app.Run();

static async Task<IResult> GetAllTodos(TodoDb db)
{
    return TypedResults.Ok(await db.Todos.Select(x => new TodoItemDTO(x)).ToArrayAsync());
}

static async Task<IResult> GetCompleteTodos(TodoDb db)
{
    return TypedResults.Ok(await db.Todos.Where(t => t.IsComplete).Select(x => new
TodoItemDTO(x)).ToListAsync());
}

static async Task<IResult> GetTodo(int id, TodoDb db)
{
    return await db.Todos.FindAsync(id)
        is Todo todo
            ? TypedResults.Ok(new TodoItemDTO(todo))
            : TypedResults.NotFound();
}

static async Task<IResult> CreateTodo(TodoItemDTO todoItemDTO, TodoDb db)
{
    var todoItem = new Todo
    {
        IsComplete = todoItemDTO.IsComplete,
        Name = todoItemDTO.Name
    };

    db.Todos.Add(todoItem);
    await db.SaveChangesAsync();

    todoItemDTO = new TodoItemDTO(todoItem);

    return TypedResults.Created($"/todoitems/{todoItem.Id}", todoItemDTO);
}

static async Task<IResult> UpdateTodo(int id, TodoItemDTO todoItemDTO, TodoDb db)
{
    var todo = await db.Todos.FindAsync(id);

    if (todo is null) return TypedResults.NotFound();

    todo.Name = todoItemDTO.Name;
    todo.IsComplete = todoItemDTO.IsComplete;

    await db.SaveChangesAsync();

    return TypedResults.NoContent();
}

static async Task<IResult> PatchTodo(int id, TodoPatchDto inputTodo, TodoDb db)
{
    var todo = await db.Todos.FindAsync(id);

    if (todo is null) return TypedResults.NotFound();

    if (inputTodo.Name is not null) todo.Name = inputTodo.Name;
    if (inputTodo.IsComplete is not null) todo.IsComplete = inputTodo.IsComplete.Value;
```

```
    await db.SaveChangesAsync();

    return TypedResults.NoContent();
}

static async Task<IResult> DeleteTodo(int id, TodoDb db)
{
    if (await db.Todos.FindAsync(id) is Todo todo)
    {
        db.Todos.Remove(todo);
        await db.SaveChangesAsync();
        return TypedResults.NoContent();
    }

    return TypedResults.NotFound();
}
```

- Run the app and verify you can post and get all fields except the `Secret` field.

Troubleshooting with the completed sample

If you run into a problem you can't resolve, compare your code to the completed project. View or download completed project (how to download).

Next steps

- Configure JSON serialization options.
- Handle errors and exceptions: The developer exception page is enabled by default in the `Development` environment for Minimal API apps. For information about how to handle errors and exceptions, see Handle errors in ASP.NET Core APIs.
- For an example of testing a Minimal API app, see this GitHub sample.
- OpenAPI support in Minimal APIs.
- Quickstart: Publish to Azure.
- Organizing ASP.NET Core Minimal APIs.

Learn more

See Minimal APIs quick reference